

Appendix C. Kodlistor

Detta appendix innehåller diverse kompletta kodlistor för klasser och program som skrivits i de olika kapitlen.

Klassen `vector`

Denna klass användes i [Kapitel 21](#). Definitionsfilen `Vector.h`:

```
#ifndef VECTOR_H
#define VECTOR_H

#include <iostream>
#include <math.h>

class Vector {
public:
    // konstruktörer
    Vector ();
    Vector (const Vector & V);
    Vector (const float X, const float Y, const float Z);

    // accessera individuella komponenter
    void setX (const float X) { m_X = X; };
    void setY (const float Y) { m_Y = Y; };
    void setZ (const float Z) { m_Z = Z; };
    float x () const { return m_X; };
    float y () const { return m_Y; };
    float z () const { return m_Z; };

    // get the length of the vector
    float length () const;

    // överlagrade operatorer
    Vector operator+ (const Vector & V) const;
    void operator+= (const Vector & V);
    Vector operator- (const Vector & V) const;
    Vector operator- () const;
    Vector operator* (const float S) const;
    bool operator== (const Vector & V) const;
    bool operator!= (const Vector & V) const;
    Vector & operator= (const Vector & V);

    // friend-funktioner
    friend Vector operator* (const float S, const Vector & V);
    friend ostream & operator<< (ostream & Stream, const Vector & V);
    friend istream & operator>> (istream & Stream, Vector & V);

private:
    // x, y och z-komponenterna
    float m_X;
    float m_Y;
    float m_Z;
};
```

```
#endif // VECTOR_H
```

Implementationsfilen Vector.cpp:

```
#include "Vector.h"

// tom konstruktor
Vector::Vector () {
    // nollställ alla medlemmar
    m_X = 0;
    m_Y = 0;
    m_Z = 0;
}

// copy-konstruktor
Vector::Vector (const Vector & V) {
    // kopiera data från 'V'
    m_X = V.m_X;
    m_Y = V.m_Y;
    m_Z = V.m_Z;
}

// skapa från separata värden
Vector::Vector (const float X, const float Y, const float Z) {
    // spara värden i medlemmar
    m_X = X;
    m_Y = Y;
    m_Z = Z;
}

// beräkna längden av vektorn
float Vector::length () const {
    // använd Pythagoras
    return sqrt ( m_X * m_X + m_Y * m_Y + m_Z * m_Z );
}

// överlagrade operatör '+'
Vector Vector::operator+ (const Vector & V) const {
    Vector Result;

    // sätt den nya vektorns värden
    Result.setX ( m_X + V.m_X );
    Result.setY ( m_Y + V.m_Y );
    Result.setZ ( m_Z + V.m_Z );

    // returnera den nya vektorn
    return Result;
}

// överlagrade '+='
void Vector::operator+= (const Vector & V) {
    m_X += V.m_X;
    m_Y += V.m_Y;
    m_Z += V.m_Z;
}

// överlagrade binära operatör '-'
Vector Vector::operator- (const Vector & V) const {
    Vector Result;

    // sätt den nya vektorns värden
    Result.setX ( m_X - V.m_X );
```

```
Result.setY ( m_Y - V.m_Y );
Result.setZ ( m_Z - V.m_Z );

// returnera den nya vektorn
return Result;
}

// överlagrade unära operatorn '-'
Vector Vector::operator- () const {
    Vector Result;

    // sätt den nya vektorns värden
    Result.setX ( -m_X );
    Result.setY ( -m_Y );
    Result.setZ ( -m_Z );

    // returnera den nya vektorn
    return Result;
}

// överlagrade binära '*'
Vector Vector::operator* (const float S) const {
    Vector Result;

    // sätt den nya vektorns värden
    Result.setX ( m_X * S );
    Result.setY ( m_Y * S );
    Result.setZ ( m_Z * S );

    // returnera den nya vektorn
    return Result;
}

// överlagrade operatorn '=='
bool Vector::operator== (const Vector & V) const {
    // jämför V1 och denna vektor
    if ( m_X == V.m_X && m_Y == V.m_Y && m_Z == V.m_Z ) {
        // vektorerna är lika
        return true;
    }

    // de är olika
    return false;
}

// överlagrade operatorn '!='
bool Vector::operator!= (const Vector & V) const {
    // använd negerad jämförelse
    return ! ( *this == V );
}

// överlagrade '='
Vector & Vector::operator= (const Vector & V) {
    // tilldelas vi värdet av oss själva?
    if ( this == &V ) {
        // jep, gör inget i så fall
        return *this;
    }

    // utför tilldelning
    m_X = V.m_X;
    m_Y = V.m_Y;
    m_Z = V.m_Z;
```

```
// returnera oss själva
return *this;
}

// extern funktion för överlagrad '*'
Vector operator* (const float S, const Vector & V) {
    // använd tidigare överlagrad operator '*'
    return V * S;
}

// extern funktion för överlagrad '<<'
ostream & operator<< (ostream & Stream, const Vector & V) {
    // skriv ut till den stream vi fått som parameter
    Stream << V.x () << ' ' << V.y () << ' ' << V.z () << ' ';

    // returnera streamen
    return Stream;
}

// extern funktion för överlagrad '>>'
istream & operator>> (istream & Stream, Vector & V) {
    // skriv ut till den stream vi fått som parameter
    Stream >> V.m_X >> V.m_Y >> V.m_Z;

    // returnera streamen
    return Stream;
}
```

[Förutgående](#)
I/O på låg nivå

[Hem](#)

[Nästa](#)
Referenser